5.1 The Concept of an Algorithm

An Informal Review

The **machine cycle** is a fundamental **algorithm executed by the CPU:**

Loop condition:

Continue until a halt instruction is encountered

Steps:

Fetch – get instruction from memory

Decode – interpret the instruction

Execute – perform the operation

The Formal Definition of an Algorithm

Algorithm: An **ordered set** of **unambiguous**, **executable** steps that defines a **terminating process**.

Ordered

Steps must be ordered

Execution structure must be well-defined

Not necessarily a simple sequence

-Examples: parallel algorithms → multiple threads

circuits (flip-flop) → ordered by cause and effect

Executable / Effective

Each step must be effective

Meaning: doable in finite time

Unambiguous

Steps must be precise and clear

Each step must be uniquely determined

No creative thinking required

Terminating

Algorithm must terminate

Must produce a final result

The Abstract Nature of Algorithms

An algorithm is abstract, while its representation is concrete.

A single algorithm can have multiple representations:

Example: Celsius → Fahrenheit

Algebraic formula:

F = (9/5)C + 32

Verbal description:

“Multiply by 9/5 and add 32”

Circuit implementation

Algorithm vs Program vs Process

Algorithm: Abstract procedure/Conceptual

Program: A representation of an algorithm/ Usually written for computers

Process: The execution of a program/execution of an algorithm

1. Summarize the distinctions between a process, an algorithm, and a program.

Algorithm: Abstract procedure/Conceptual

Program: A representation of an algorithm/ Usually written for computers

Process: The execution of a program

Program represents an algorithm, and a process executes it.

2. Give some examples of algorithms with which you are familiar. Are they really algorithms in the precise sense?

Examples of algorithms include:

- Cooking recipes
- Directions for navigating a city
- Instructions for operating a washing machine
- Long division algorithm
- The Euclidean algorithm

However, not all of these are algorithms in the precise sense:

- Many everyday examples are informal and may be ambiguous, non-terminating, or not fully executable
- Therefore, they do not always satisfy the formal definition of an algorithm

3. Identify some points of vagueness in our informal definition of an algorithm introduced in Section 0.1 of the introductory chapter.

The informal definition of an algorithm as "a set of steps for performing a task" is vague because it does not specify:

- Whether the steps must be unambiguous
- Whether they must be executable (effective)
- Whether the process must terminate
- Whether the steps must be ordered

4. In what sense do the steps described by the following list of instructions fail to constitute an algorithm?

Step 1. Take a coin out of your pocket and put it on the table.

Step 2. Return to Step 1.

Terminating

The steps create an infinite loop

The process never produces a final result

5.2 Algorithm Representation

Primitives

Primitive(原语)

A **primitive** is a **basic building block** used to construct algorithm representations.

primitive 就是：最基础、不能再拆的操作

Each primitive has:

- syntax (its form)
- semantics (its meaning)

Primitives help to:

- eliminate ambiguity
- ensure a uniform level of detail
- make algorithm representations precise

Figure 5.2 Folding a bird from a square piece of paper

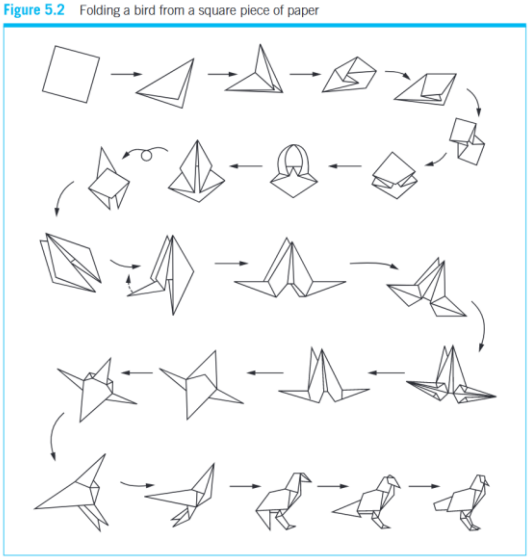
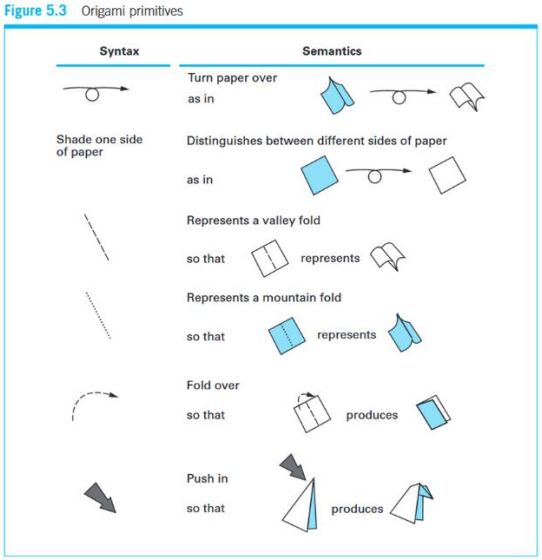


Figure 5.3 Origami primitives



programming language

A collection of **primitives** together with **rules** for combining them.

Machine-level primitives: very detailed/ directly executable/ tedious to use

High-level primitives: more abstract/ easier for humans/ built from lower-level primitives

Pseudocode（伪代码）

Pseudocode is an **informal notational system** used to express algorithms during the development process.

It is less formal than a programming language

It borrows syntax and semantic structures from programming languages

Assignment Structure（赋值）

name = expression

Selection Structure（if-else）

```
if (condition):
    activity
else:
    activity
```

Repetition Structure（while 循环）

```
while (condition):
```

activity

Indentation（缩进）

Indentation is used to define structure

In Python-like pseudocode, it is essential

Functions（函数）

```
def name():
    statements
name()
```

Parameters（参数）

```
def Sort(List):
Sort(the organization's membership list)
```

Naming Items in Programs

Avoid long multiword names in natural language/Use single contiguous identifiers

-snake_case

estimated_arrival_time

-Pascal casing 每个单词首字母大写

EstimatedArrivalTime

-camel casing 第一个小写，后面大写

estimatedArrivalTime

1. A primitive in one context might turn out to be a composite of primitives in another. For instance, our while statement is a primitive in our pseudocode, yet it is ultimately implemented as a composite of machine-language instructions. Give two examples of this phenomenon in a non-computer setting.

In cooking, the instruction "bake a cake" may be considered a primitive at a high level, but it is actually composed of smaller steps such as mixing ingredients, setting the oven temperature, and timing the baking process.

In driving, the instruction "drive to the destination" may be treated as a primitive, but it consists of many smaller actions such as steering, accelerating, braking, and following traffic rules.

2. In what sense is the construction of functions the construction of primitives?

The construction of functions is the construction of primitives because a function encapsulates a sequence of steps into a single unit.

函数的构造之所以被称为原语的构造，是因为函数将一系列步骤封装成一个整体。

Once defined, the function can be used as a basic building block (primitive) in other parts of an algorithm, hiding its internal complexity.

一旦函数被定义，它就可以作为一个基本操作在算法的其他部分中使用，从而隐藏其内部的复杂性。

3. The Euclidean algorithm finds the greatest common divisor of two positive integers X and Y by the following process: As long as the value of neither X nor Y is zero, assign the larger the remainder of dividing the larger by the smaller. The greatest common divisor, if it exists, will be the remaining non-zero value. Express this algorithm in our pseudocode.

```
while (X != 0 and Y != 0):
```

```
    if (X > Y):
        X = X % Y
    else:
        Y = Y % X
if (X == 0):
    GCD = Y
else:
    GCD = X
```

4. Describe a collection of primitives that are used in a subject other than computer programming.

A collection of primitives can be found in many fields.

For example, in mathematics:

addition

subtraction

multiplication

division

These operations serve as basic building blocks for more complex calculations.

Another example is in origami, where primitives include basic folds that are combined to create more complex structures.

5.3 Algorithm Discovery

The development of a program consists of two main activities:

1. Discovering the underlying algorithm
2. Representing that algorithm as a program

The Art of Problem Solving

Polya's Four Phases:

1. Understand the problem (Understand the problem)
2. Devise a plan (Get an idea of an algorithm)
3. Carry out the plan (Formulate and implement the algorithm)
4. Evaluate the solution (Evaluate the program)

These phases are not steps to be followed, but phases that occur during problem solving.

Non-linear Process（非线性过程）

Software engineers prefer a complete understanding before implementation to avoid costly mistakes. However, true understanding often comes only after attempting a solution.

Getting a Foot in the Door

Problem-Solving Approaches: Working Backward（逆向思考）/ Solve Related Problems（类比问题）

stepwise refinement

Stepwise refinement is a technique in which a problem is broken into smaller subproblems, and each subproblem is further divided until all parts are manageable.

top-down methodology progresses from general to specific.

bottom-up methodology starts from specific components and builds toward a complete system.

Although different, top-down and bottom-up approaches often complement each other in practice.

1. a. Find an algorithm for solving the following problem: Given a positive integer n, find the list of positive integers whose product is the largest among all the lists of positive integers whose sum is n. For example, if n is 4, the desired list is 2, 2 because 2 * 2 is larger than 1 * 1 * 1 * 1, 2 * 1 * 1, and 3 * 1. If n is 5, the desired list is 2, 3.
b. What is the desired list if n = 2001?
c. Explain how you got your foot in the door.
把整数拆成尽可能多的 3，乘积最大。
a. Algorithm (算法思路)

1. 尽量把 n拆成尽可能多的 3。
2. 如果拆出剩余 1，用 2 + 2 替代。
3. 如果拆出剩余 2，保留。
4. 输出列表。

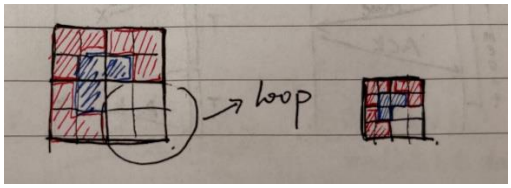
这是 stepwise refinement 的体现：把大问题拆成“拆 3、处理剩余 1、处理剩余 2”几个小步骤。
b. n = 2001 的列表

- 2001 除以 3 得 667 余 0
- 所以列表就是 667 个 3

c. Foot in the door

- 从小例子（n=4,5,6,7）开始分析，发现规律：多拆 3 最优
- 这就是“先找到突破口”（foot in the door），然后推广到大 n。

2. a. Suppose we are given a checkerboard consisting of 2^n rows and 2^n columns of squares, for some positive integer n, and a box of L-shaped tiles, each of which can cover exactly three squares on the board. If any single square is cut out of the board, can we cover the remaining board with tiles such that tiles do not overlap or hang off the edge of the board?
b. Explain how your solution to (a) can be used to show that 2^2n - 1 is divisible by 3 for all positive integers n.
c. How are (a) and (b) related to Polya's phases of problem solving?



a. Can we cover the board?
是的，可以。
用 递归法：把 $2^n \times 2^n$棋盘分成 4 个 $2^{n-1} \times 2^{n-1}$子棋盘。
在每个子棋盘放置一个 L-shaped tile 覆盖中心的三个格子。
递归处理剩下的子棋盘，直到 n=1。
b. 证明 2^(2n)-1 可被 3 整除
棋盘剩余格子数 = $2^{2n} - 1$
每块 L-shaped tile 覆盖 3 个格子
因为棋盘可以完全铺满，所以 $2^{2n} - 1$必须能被 3 整除。
c. 与 Polya phases 的关系

Phase 1（Understand the problem）：理解棋盘大小和 L-tile 覆盖规则
Phase 2（Devise a plan）：想到递归分割的方法
Phase 3（Carry out the plan）：实际设计递归覆盖
Phase 4（Evaluate）：验证能覆盖整个棋盘且符合 3 的倍数性质

3. Decode the following message, then explain how you got your foot in the door.
Pdeo eo pda yknnayp wjosan.
This is the correct answer
We recognize patterns such as “pda” and guess it means “the.”

4. Would you be following a top-down methodology if you attempted to solve a picture puzzle merely by pouring the pieces out on a table and trying to piece them together? Would your answer change if you looked at the puzzle box to see what the entire picture was supposed to look like?
如果只是把拼图倒在桌上随便拼 → 不是 top-down，是 bottom-up 方法（从小块到整体）
如果先看盒子上的完整图片 → 是 top-down，从整体结构指导局部拼图

5.4 Iterative Structures

iterative structures: The main goal is to study repetitive structures in algorithms, where a set of instructions is executed multiple times.

The Sequential Search Algorithm

sequential search

Figure 5.6 The sequential search algorithm in pseudocode

```
def Search(List, TargetValue):
  if (List is empty):
    Declare search a failure.
  else:
    Select the first entry in List to be TestEntry.
    while (TargetValue > TestEntry and
        there remain entries to be considered):
      Select the next entry in List as TestEntry.
    if (TargetValue == TestEntry):
      Declare search a success.
    else:
      Declare search a failure.
```

书上的例子默认 list 从小到大排好序

Loop Control

Loop

A loop is an iterative structure in which a collection of instructions, called the **body 循环体**, is executed repeatedly under the direction of a control process.

Body

The body of a loop contains the instructions that actually perform the task of interest.

termination condition

The termination condition is the condition that ends the loop. A typical loop control has three components:

1. **Initialize** (初始化) – set the starting state.
2. **Test** (测试) – check whether the termination condition is met.
3. **Modify** (修改) – change the state so that the loop progresses toward termination.

flowchart

Diamond → decision/test condition

Rectangle → body/action

Figure 5.8 The while loop structure

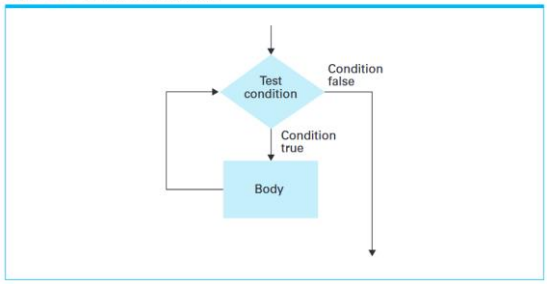
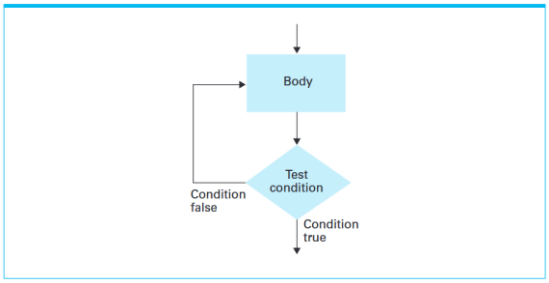


Figure 5.9 The repeat loop structure



pretest loop 先测试循环（while 循环） test occurs before the body executes.

while (condition):

    body

posttest loop 后测试循环（repeat 循环）body executes before testing termination

repeat:

    body

until (condition)

for-each Common in list processing: iterate from the first element to the last.

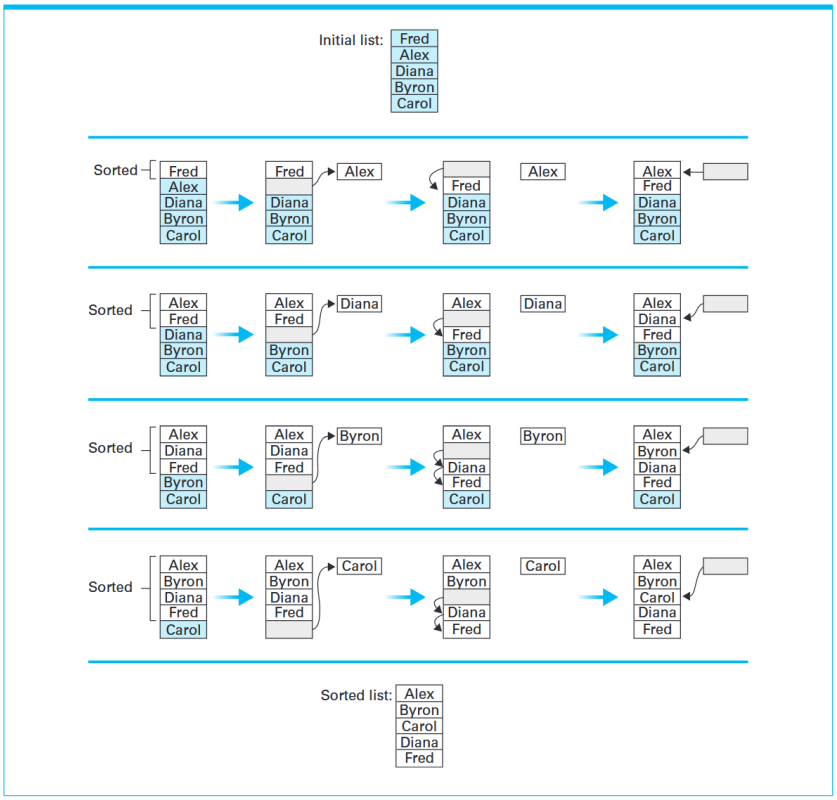
for Item in List:

    body

The Insertion Sort Algorithm

Insertion sort solves the problem of sorting a list **in-place**, meaning the entries are rearranged within the same list without using additional storage.

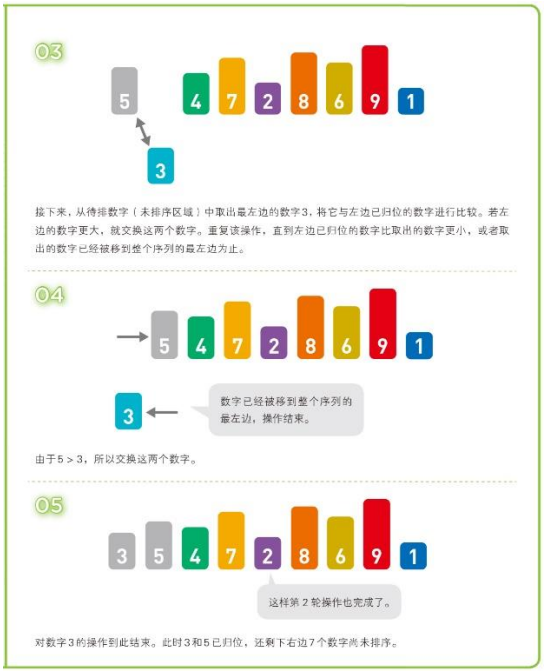
Figure 5.10 Sorting the list Fred, Alex, Diana, Byron, and Carol alphabetically



insertion sort

Figure 5.11 The insertion sort algorithm expressed in pseudocode

```
def Sort (List):
  N = 2
  while (the value of N does not exceed the length of List):
    Select the Nth entry in List as the pivot entry.
    Move the pivot entry to a temporary location leaving
      a hole in List.
    while (there is a name above the hole and that name
        is greater than the pivot):
      Move the name above the hole down into the hole
        leaving a hole above the name.
    Move the pivot entry into the hole in List.
    N = N + 1
```



1. Modify the sequential search function in Figure 5.6 to allow for lists that are not sorted.

```
def Search(List, TargetValue):
    if (List is empty):
        Declare search a failure.
    else:
        Select the first entry in List to be TestEntry.
        while (TargetValue != TestEntry and
            there remain entries to be considered):
            Select the next entry in List as TestEntry.
        if (TargetValue == TestEntry):
            Declare search a success.
        else:
            Declare search a failure.
```

2. **Convert the pseudocode routine**

```
Z=0
X=1
while (X < 6):
    Z=Z+X
    X=X+1
```

to an equivalent routine using a repeat statement.

```
Z=0
X=1
repeat
    Z=Z+X
    X=X+1
until (X>=6)
```

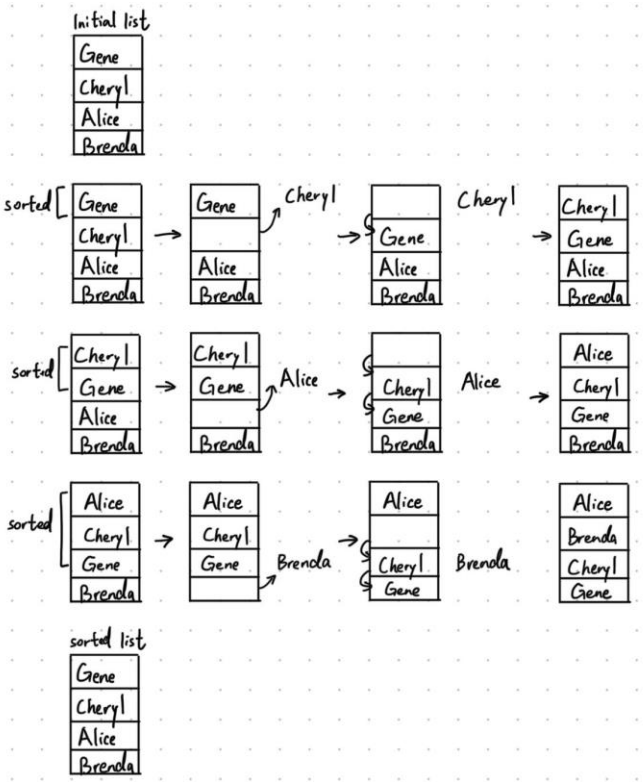
while (条件) ↔ repeat until (条件取反)

3. **Some of the popular programming languages today use the syntax**
   while (. . .) do (. . .)
   to represent a pretest loop and the syntax
   do (. . .) while (. . .)
   to represent a posttest loop. Although elegant in design, what problems could result from such similarities?

   do (. . .) while (. . .) will do one more time do (. . .) than while (. . .) do (. . .)

   a do-while loop always executes at least once, while a while loop may not execute at all

4. **Suppose the insertion sort as presented in Figure 5.11 was applied to the list Gene, Cheryl, Alice, and Brenda. Describe the organization of the list at the end of each execution of the body of the outer while structure.**



5. Why would we not want to change the phrase "greater than" in the while statement in Figure 5.11 to "greater than or equal to"?

If equal, there is no need to change.

If "greater than or equal to" is used, elements equal to the pivot would also be moved.

This would destroy the relative order of equal elements, making the algorithm unstable.

stable = 相同的东西不换位置

unstable = 相同的东西也可能被乱换

6. **A variation of the insertion sort algorithm is the selection sort. It begins by selecting the smallest entry in the list and moving it to the front. It then selects the smallest entry from the remaining entries in the list and moves it to the second position in the list. By repeatedly selecting the smallest entry from the remaining portion of the list and moving that entry forward, the sorted version of the list grows from the front of the list, while the back portion of the list consisting of the remaining unsorted entries shrinks. Use our pseudocode to express a function similar to that in Figure 5.11 for sorting a list using the selection sort algorithm.**

Set N to 1.

While (N is less than the length of List):

    Select the entry at position N to be the current position.

    Search the portion of the list from position N through the end
    to find the position of the smallest entry.
    Call this position MinPosition.

    If (MinPosition is not N):
        Exchange the entries at positions N and MinPosition.

    Increase N by 1.

7. Another well-known sorting algorithm is the bubble sort. It is based on the process of repeatedly comparing two adjacent names and interchanging them if they are not in the correct order relative to each other. Let us suppose that the list in question has n entries. The bubble sort would begin by comparing (and possibly interchanging) the entries in positions n and n – 1. Then, it would consider the entries in positions n – 1 and n – 2, and continue moving forward in the list until the first and second entries in the list had been compared (and possibly interchanged). Observe that this pass through the list will pull the smallest entry to the front of the list. Likewise, another such pass will ensure that the next to the smallest entry will be pulled to the second position in the list. Thus, by making a total of n – 1 passes through the list, the entire list will be sorted. (If one watches the algorithm at work, one sees the small entries bubble to the top of the list—an observation from which the algorithm gets its name.) Use our pseudocode to express a function similar to that in Figure 5.11 for sorting a list using the bubble sort algorithm.

Set N to the length of List.

While (N is greater than 1):

    Set M to 1.

    While (M is less than N):

        Compare the entries at positions M and M + 1.

        If (the entry at position M is greater than the entry at position M + 1):
            Exchange these two entries.

        Increase M by 1.

    Decrease N by 1.

5.5 Recursive Structures

Recursive structures provide an alternative to loops for repeating activities.
A loop repeats a set of instructions after completing them, while recursion repeats the instructions as a subtask of itself.

The Binary Search Algorithm

Binary search is an algorithm used to search for a target value in a sorted list.
The algorithm repeatedly divides the list into two halves and restricts the search to one half.

Figure 5.12 Applying our strategy to search a list for the entry John

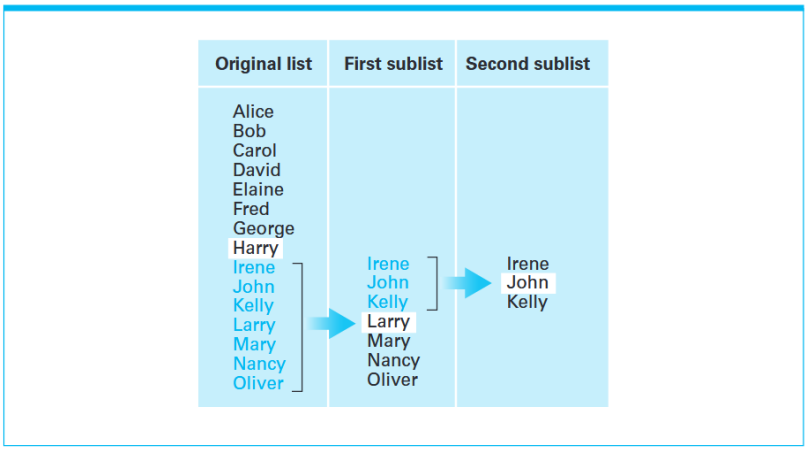


Figure 5.13 A first draft of the binary search technique

```
if (List is empty):
  Report that the search failed.
else:
  TestEntry = the "middle" entry in the List
  if (TargetValue == TestEntry):
    Report that the search succeeded.
  if (TargetValue < TestEntry):
    Search() the portion of List preceding TestEntry for TargetValue,
      and report the result of that search.
  if (TargetValue > TestEntry):
    Search() the portion of List following TestEntry for TargetValue,
      and report the result of that search.
```

Figure 5.14 The binary search algorithm in pseudocode

```
def Search(List, TargetValue):
  if (List is empty):
    Report that the search failed.
  else:
    TestEntry = the "middle" entry in List
    if (TargetValue == TestEntry):
      Report that the search succeeded.
    if (TargetValue < TestEntry):
      Sublist = portion of List preceding
        TestEntry
      Search(Sublist, TargetValue)
    if (TargetValue > TestEntry):
      Sublist = portion of List following
        TestEntry
      Search(Sublist, TargetValue)
```

Figure 5.15 Recursively Searching

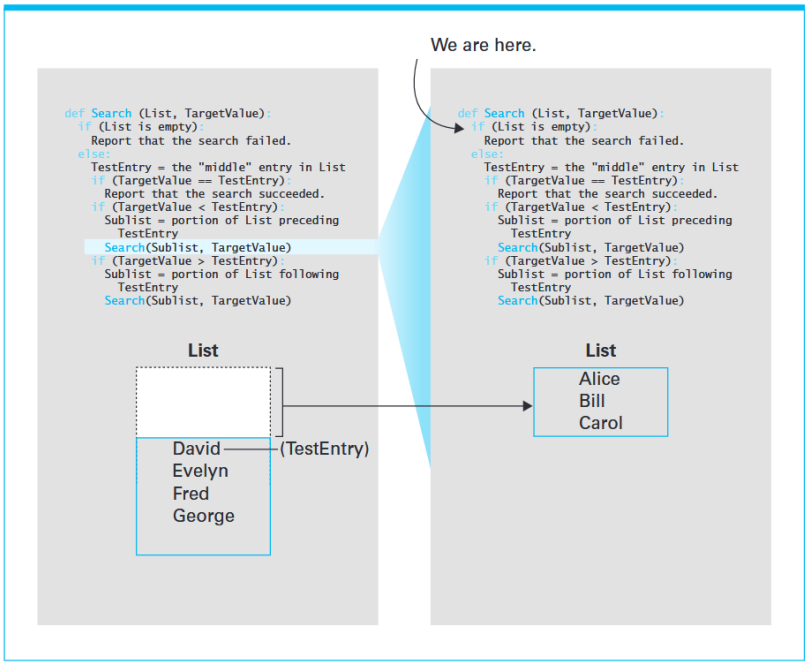


Figure 5.16 Second Recursive Search, First Snapshot

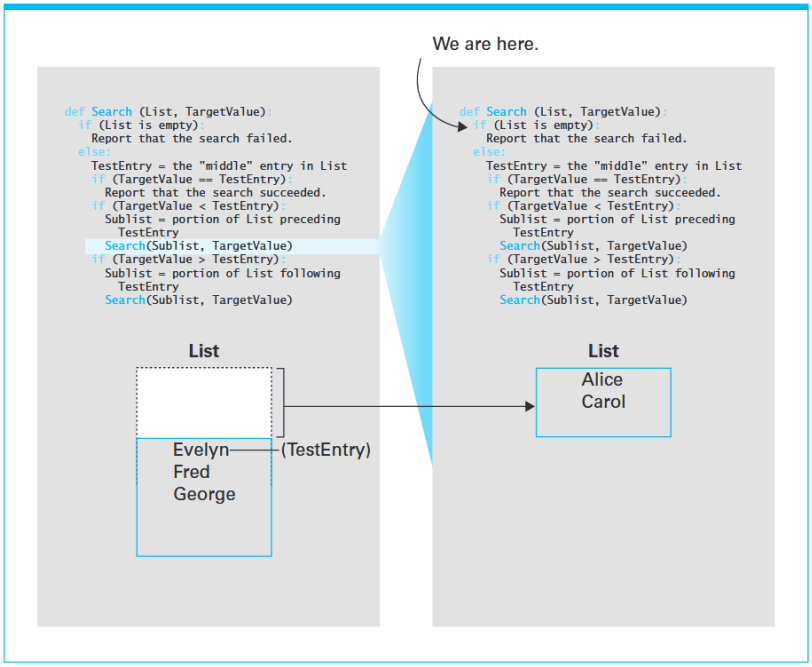
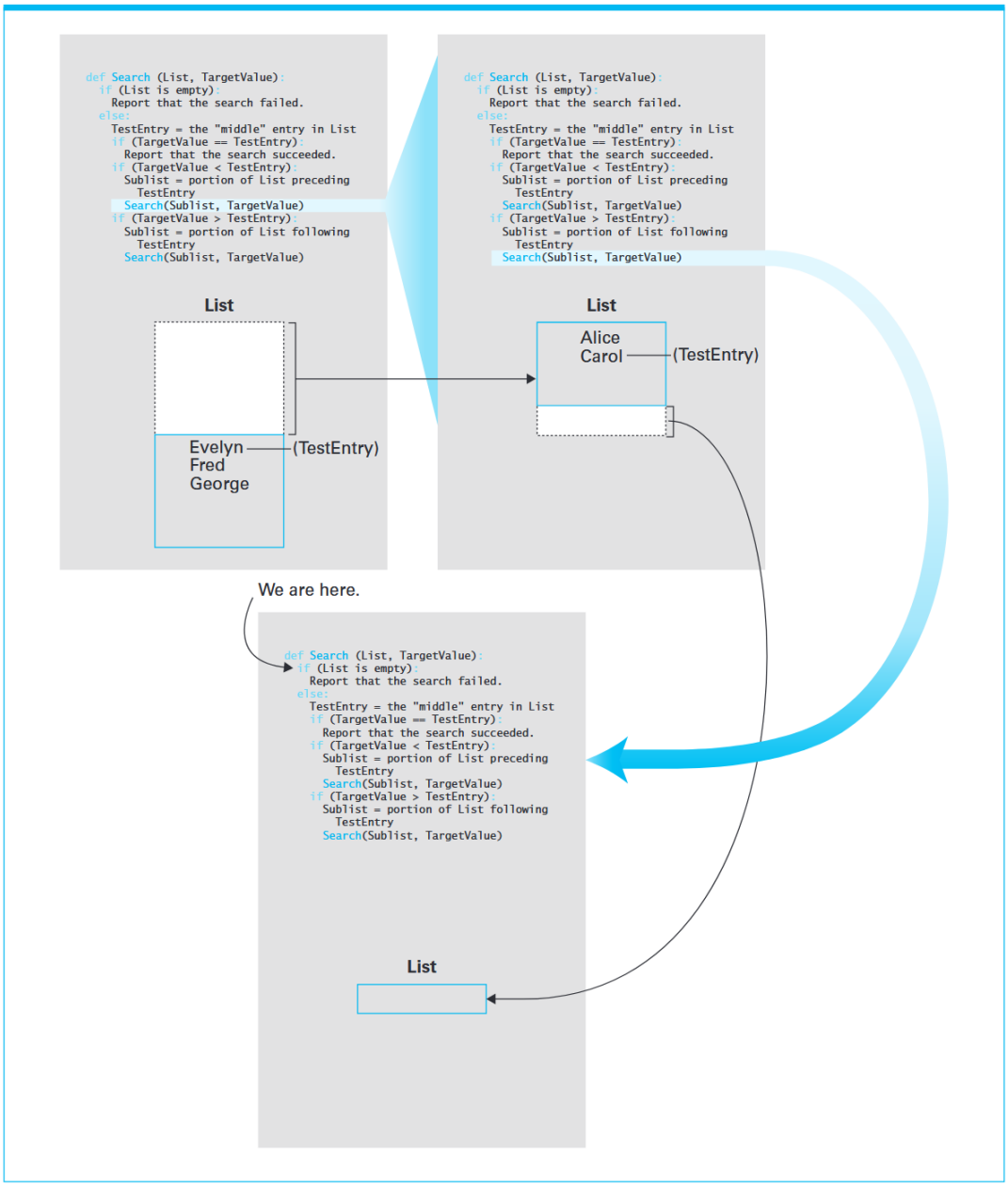


Figure 5.17 Second Recursive Search, Second Snapshot



Recursive Control

Recursive control is a technique for implementing repetition by allowing a function to call itself.

Activation（函数激活）: Each execution of a recursive function is called an activation.

Three Elements of Control

(1) Initialization（初始化） Initialization provides the initial problem to be solved.

(2) Modification（修改） Modification reduces the problem to a smaller subproblem.

(3) Termination Test（终止条件） A test is used to determine when the recursion should stop.

The **base case 基本情况** (or **degenerative case 退化情况**) is the condition under which recursion stops.

01

还是来试试查找数字6吧。

02

首先找到数组中间的数字，此处为5。

03

将5和要查找的数字6进行比较。

04

把不需要的数字移出查找范围。

05

在剩下的数组中找到中间的数字，此处为7。

06

比较7和6。

07

把不需要的数字移出查找范围。

08

在剩下的数组中找到中间的数字，此处为6。

09

6=6，成功找到目标数字。

1. What names are interrogated by the binary search (Figure 5.14) when searching for the name Joe in the list Alice, Brenda, Carol, Duane, Evelyn, Fred, George, Henry, Irene, Joe, Karl, Larry, Mary, Nancy, and Oliver?

Alice, Brenda, Carol, Duane, Evelyn, Fred, George, Henry, Irene, Joe, Karl, Larry, Mary, Nancy, Oliver

Alice, Brenda, Carol, Duane, Evelyn, Fred, George, Henry, Irene, Joe, Karl, Larry, Mary, Nancy, Oliver

Alice, Brenda, Carol, Duane, Evelyn, Fred, George, Henry, Irene, Joe, Karl, Larry, Mary, Nancy, Oliver

Henry -> Larry -> Joe

2. What is the maximum number of entries that must be interrogated when applying the binary search to a list of 200 entries? What about a list of 100,000 entries?

Binary search requires at most $\lceil \log_2(n) \rceil$ comparisons.

n = 200 At most 8 entries.

$2^7 = 128 < 200 < 256 = 2^8$

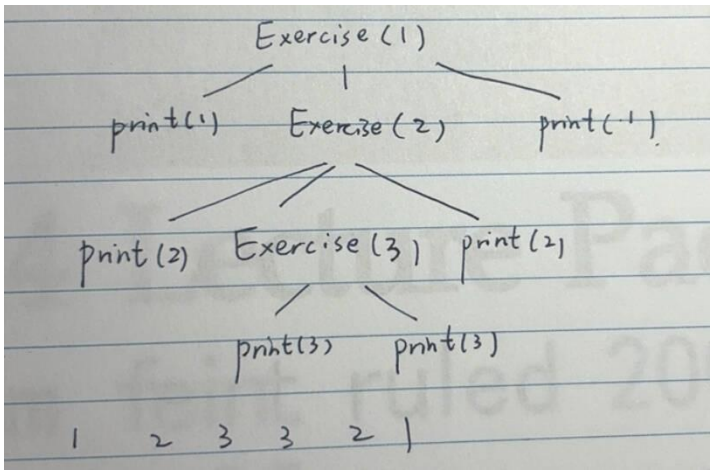
n = 100,000At most 17 entries.

$2^{16} = 65{,}536 < 100{,}000 < 131{,}072 = 2^{17}$

3. What sequence of numbers would be printed by the following recursive function if we started it with N assigned the value 1?

```
def Exercise (N):
    print(N)
    if (N < 3):
        Exercise(N + 1)
    print(N)
```

123321



4. What is the termination condition in the recursive function of question 3?

N>=3

5.6 Efficiency and Correctness

Algorithm Efficiency Algorithm efficiency refers to the amount of resources (such as time or space) required by an algorithm.

Example:

(1) Sequential Search（顺序查找）O(n)

(2) Binary Search（二分查找）O(log n)

Algorithm Analysis studies the resources required by algorithms.

Types of Analysis

(1) Best Case The minimum amount of work required by the algorithm.

(2) Worst Case The maximum amount of work required.

(3) Average Case The expected amount of work over many executions.

Algorithm analysis is often expressed in terms of n, the size of the input.

Example: Insertion Sort

- Best Case: If the list is already sorted, each element requires only one comparison.

Total comparisons: n − 1

O(n)

- Worst Case: If the list is in reverse order, each element must be compared with all previous elements.

Total comparisons:

$1 + 2 + 3 + \ldots + (n - 1) = (1/2)(n^2 - n)$

$O(n^2)$

Figure 5.18 Applying the insertion sort in a worst-case situation.

	Comparisons made for each pivot				
Initial list	1st pivot	2nd pivot	3rd pivot	4th pivot	Sorted list
Elaine David Carol Barbara Alfred	1 Elaine David Carol Barbara Alfred	3 David 2 Elaine Carol Barbara Alfred	6 Carol 5 David 4 Elaine Barbara Alfred	10 Barbara 9 Carol 8 David 7 Elaine Alfred	Alfred Barbara Carol David Elaine

- Average Case of Insertion Sort

In the average case, each pivot is compared to about half of the preceding entries.

The total number of comparisons is approximately $(1/4)(n^2 - n)$.

Time Approximation（时间近似） The number of comparisons provides an approximation of the time required to execute the algorithm.

Shape of Graphs

The efficiency of an algorithm is reflected by the shape of its time-growth graph.

- Linear（线性）

Straight line 直线增长

- Quadratic（二次）

Parabolic curve (n^2) 抛物线（n^2）

- Logarithmic（对数）

Logarithmic curve (log n) 对数曲线

Figure 5.19 Graph of the worst-case analysis of the insertion sort algorithm

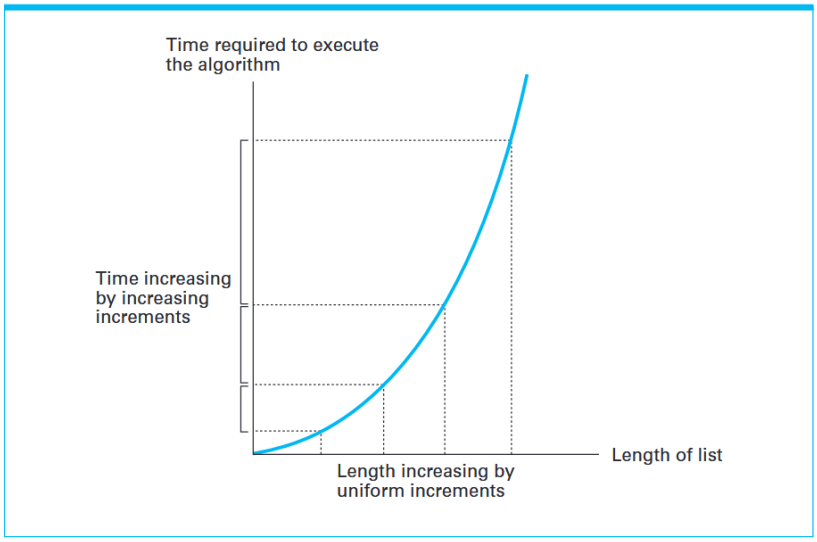
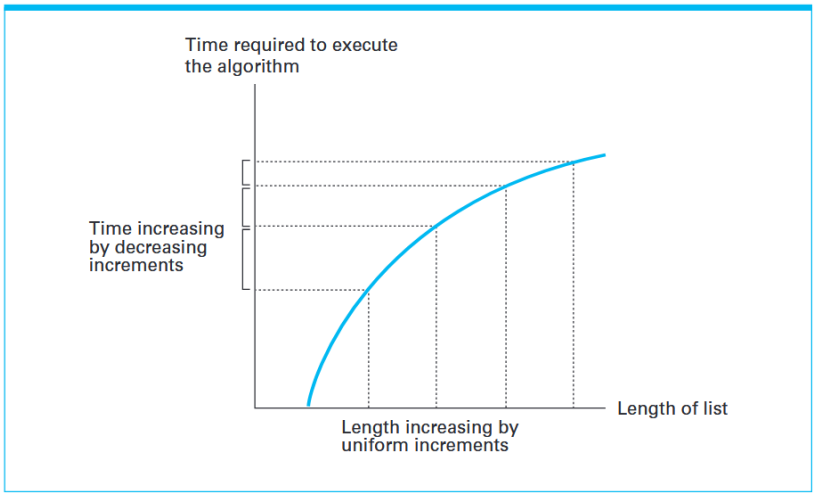


Figure 5.20 Graph of the worst-case analysis of the binary search algorithm



big-theta notation（大 Θ 记号）

Big-theta notation is used to classify algorithms according to their growth rate.

Software Verification

Software verification is the process of ensuring that a program is correct.

Preconditions（前置条件）

Preconditions are conditions assumed to be true before a program begins execution.

Assertions（断言）

Assertions are statements that are known to be true at specific points in a program.

Propagation of Assertions（断言传播）

Assertions are updated as the program executes.

If Y ≠ 0 before X = Y, then X ≠ 0 after the assignment.

Postconditions（后置条件）

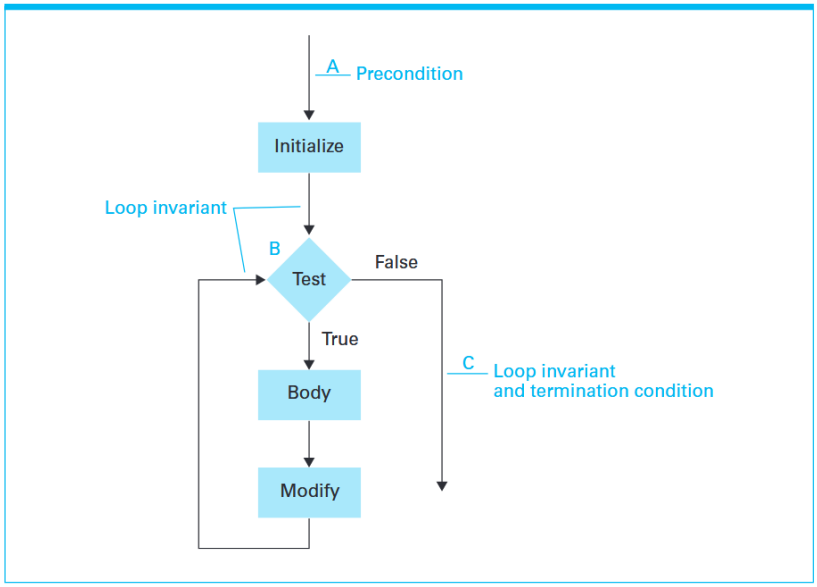
Postconditions are the desired conditions that should be true after program execution.

They describe the expected output of the program.

loop invariant（循环不变式）

A loop invariant is an assertion that is true every time the loop condition is tested.
It helps prove that the loop behaves correctly.
If the loop terminates, both the loop invariant and the termination condition hold.

Figure 5.23 The assertions associated with a typical while structure



Example: Insertion Sort（插入排序例子）
Loop invariant: The first N−1 elements are sorted.
Termination condition: N exceeds the length of the list.
Conclusion: The entire list is sorted.
Formal Verification（形式化验证）
Formal verification uses logic to prove that a program satisfies its specifications.
Limitations of Testing（测试的局限）
Testing can only show that a program works for specific cases.

1. Suppose we find that a machine programmed with our insertion sort algorithm requires an average of one second to sort a list of 100 names. How long do you estimate it takes to sort a list of 1,000 names? How about 10,000 names?
average 1/4*n(n-1)
100 1/4*100(100-1)=2475 1s
1000 1/4*1000(1000-1)=249750 101s 约 100s
10000 1/4*10000(10000-1)=24997500 10100s 约 10000s

2. Give an example of an algorithm in each of the following classes: Θ(log2 n), Θ(n), and Θ(n^2).
Q(log2 n) binary search
Q(n) sequencial find
Q(n^2) bubble sort/ insert sort

3. List the classes Θ(n^2), Θ(log2 n), Θ(n), and Θ(n^3) in decreasing order of efficiency.
Θ(log2 n) > Θ(n) > Θ(n^2) > Θ(n^3)

4. Consider the following problem and a proposed answer. Is the proposed answer correct? Why or why not?
Problem:—Suppose a box contains three cards. One of three cards is painted black on both sides, one is painted red on both sides, and the third is painted red on one side and black on the other. One of the cards is drawn from the box, and you are allowed to see one side of it. What is the probability that the other side of the card is the same color as the side you see?
Proposed answer:—One-half. Suppose the side of the card you can see is red. (The argument would be symmetric with this one if the side were black.) Only two cards among the three have a red side. Thus the card you see must be one of these two. One of these two cards is red on the other side, while the other is black. Thus the card you can see is just as likely to be red on the other side as it is to be black.
Incorrect The reasoning ignores that the cards do not have the same number of red sides.
2/3

5. The following program segment is an attempt to compute the quotient (forgetting any remainder) of two positive integers (a dividend and a divisor) by counting the number of times the divisor can be subtracted from the dividend before what is left becomes less than the divisor. For instance, 7/3 should produce 2 because 3 can be subtracted from 7 twice. Is the program correct? Justify your answer.

```
Count = 0
Remainder（余数） = Dividend（被除数）
repeat:
    Remainder = Remainder – Divisor
    Count = Count + 1
    until (Remainder < Divisor)
    Quotient（商） = Count.
```

Incorrect 2÷3

```
Count = 0
Remainder（余数） = Dividend（被除数）
If (Remainder >= Divisor):
    Remainder = Remainder – Divisor
    Count = Count + 1
Quotient（商） = Count.
```

6. The following program segment is designed to compute the product of two nonnegative integers X and Y by accumulating the sum of X copies of Y— that is, 3 times 4 is computed by accumulating the sum of three 4s. Is the program correct? Justify your answer.

```
Product = Y
Count = 1
while (Count < X):
    Product = Product + Y
```

Count = Count + 1

Incorrect X=0 product=0

Product = 0

Count = 0

while (Count < X):

Product = Product + Y

Count = Count + 1

7. Assuming the precondition that the value associated with N is a positive integer, establish a loop invariant that leads to the conclusion that if the following routine terminates, then Sum is assigned the value 0 + 1 + ··· + N.

Sum = 0

K=0

while (K < N):

K=K+1

Sum = Sum + K

Provide an argument to the effect that the routine does in fact terminate.

loop invariant

loop invariant: At the beginning of each iteration, Sum = 1 + 2 + ··· + K

The variable K is initialized to 0 and increases by 1 in each iteration. Since N is a positive integer and the loop condition is K < N, K will eventually reach N. At that point, the condition becomes false and the loop terminates. 变量 K 初始为 0，并且每次循环增加 1。由于 N 是正整数，且循环条件是 K < N，因此 K 最终会达到 N，此时条件不成立，循环终止。

8. Suppose that both a program and the hardware that executes it have been formally verified to be accurate. Does this ensure accuracy?

No, this does not necessarily ensure accuracy.

Even if both the program and hardware are formally verified, accuracy is not guaranteed because verification depends on the correctness of the specifications and assumptions.